Guia de fala da equipe
Corre que Derrete!

Daniel Pena, John Lucas e Lucas Oliveira
7º período de Engenharia da Computação
Professor Pedro Cândido - entrega em 16/10/2026

Como usar

As falas são sugestões para ensaiar. Usem suas palavras e expliquem o que conseguem mostrar no jogo. O roteiro prevê cerca de 10 a 12 minutos de exposição, seguidos de perguntas e 2 a 3 minutos de demonstração. As mesmas sugestões estão nas notas do PowerPoint.

Divisão sugerida

Daniel: ideia, regras, evolução e problemas encontrados.
John: estados, organização do código e colisões.
Lucas: parallax, animação, partículas e demonstração.
Os créditos e o uso de IA podem ser apresentados por quem encerrar a exposição. Essa é uma divisão de fala proposta, que vocês podem ajustar. Não representa quem programou cada módulo.

Contribuições registradas

Os três participaram das escolhas do tema, corrida lateral, obstáculos e itens, conforme relato de Daniel. Os testes no Windows e o retorno sobre problemas foram realizados por Daniel. ChatGPT/Codex geraram grande parte do código e apoiaram arte, testes e documentação. Não atribuam a uma pessoa programação ou validação que ela não realizou.

Uso de IA e a regra do professor

A seção 11 cita pesquisa, depuração, geração de ideias e revisão conforme as regras da disciplina, além de exigir registro da ferramenta, finalidade, partes afetadas e validação. O texto não autoriza expressamente geração da maior parte do código. Confirmem esse alcance com o professor, descrevendo o uso real. Alterar o estilo da documentação não resolve essa dúvida. Se ele exigir implementação mais direta da equipe, combinem com ele a adequação e o que precisará ser refeito.

Antes de apresentar

Abram o jogo no computador escolhido, confiram som, pausa e F11 e deixem a apresentação disponível também em PDF. Guardem uma cópia offline. A gravação de segurança deve ser da versão final e identificada como vídeo, sem substituir a execução ao vivo.

Falas sugeridas dos slides 1 a 4

1. Corre que Derrete! (30 segundos)

O nosso projeto é uma corrida lateral em que o entregador precisa chegar ao cliente antes que o sorvete derreta. O personagem corre sozinho, e o jogador decide quando saltar. Vamos apresentar a ideia, as técnicas gráficas e depois mostrar uma partida.

2. A ideia do jogo (50 segundos)

Escolhemos juntos o tema, a corrida lateral e os objetos que aparecem no caminho. Queríamos uma partida curta, com controles simples, para colegas e jogadores iniciantes. O sorvete deu sentido às regras: o calor reduz a barra, os impactos fazem perder conservação e o gelo ajuda a recuperar. O objetivo de projeto foi criar um jogo pequeno, mas com começo, desafio e conclusão.

3. Regras e controles (50 segundos)

A conservação começa em cem por cento. O calor tira dois pontos percentuais por segundo ativo, e bater em cone ou caixa tira vinte. O gelo recupera até quinze, respeitando o limite de cem. Por exemplo, um impacto com oitenta deixa sessenta. A rota tem mil e oitocentos metros, e a dificuldade aumenta em três trechos. Para vencer, é preciso chegar com conservação acima de zero. A pontuação soma os metros percorridos e cem pontos por gelo.

4. O caminho entre as telas (45 segundos)

O jogo usa estados para decidir o que recebe comandos e o que aparece na tela. No menu, podemos iniciar ou ler as instruções. Na corrida, podemos pausar e retomar. A partida termina no resultado, que pode ser vitória ou derrota. O resultado permite reiniciar ou voltar ao menu. As opções de áudio estão no menu e na pausa. Ao ajustar o volume durante uma pausa, a corrida continua congelada.

Falas sugeridas dos slides 5 a 7

5. Como o código se organiza (60 segundos)

O código separa os comandos, as regras e o desenho. A classe App recebe eventos e controla o estado. Run guarda a corrida e aplica as regras. Renderer desenha, e Window cuida da janela e da tela cheia. A cada repetição do loop, o jogo lê os comandos, atualiza a partida e desenha a tela. O tempo entra nos cálculos para o movimento não depender só da quantidade de quadros. A simulação usa passos fixos de um sessenta avos de segundo. Na pausa, a atualização da corrida para.

6. Salto, câmera e colisão (55 segundos)

No salto, o personagem recebe velocidade para cima. A gravidade faz ele voltar ao chão. Enquanto isso, a câmera acompanha o avanço da rota: os objetos se deslocam na tela, mas o HUD fica parado. Para colisões, o jogo usa caixas retangulares, chamadas hitboxes. O collliderect verifica quando elas se sobrepõem. Cada obstáculo pode causar dano uma vez por passagem. A proteção evita impactos seguidos por um curto intervalo. A caixa do cone é uma aproximação, não acompanha cada detalhe da sua forma.

7. Parallax no cenário (45 segundos)

O parallax cria profundidade usando velocidades diferentes. As nuvens andam devagar, os prédios ficam no meio e as fachadas próximas se movem mais rápido. São camadas de um cenário 2D. O fundo se repete para continuar a corrida sem terminar de repente. Na implementação, cada camada usa um fator aplicado à distância da câmera. A operação de módulo faz a repetição.

Falas sugeridas dos slides 8 a 10

8. O movimento do personagem (45 segundos)

A animação funciona pela troca de quadros. Existem seis poses de corrida, alternadas dez vezes por segundo. As pernas mudam de posição e dão a impressão de movimento. No salto, o jogo escolhe uma pose para a subida e outra para a descida. Os quadros ficam guardados na memória. A animação muda o desenho, enquanto a caixa de colisão mantém o mesmo tamanho.

9. Partículas e avisos (45 segundos)

Os efeitos ajudam a entender o que aconteceu. A coleta produz cristais, o impacto gera faíscas e a conservação baixa mostra gotas. Cada partícula nasce com uma posição, uma velocidade e um tempo de vida. Ela se move e desaparece ao acabar esse tempo. O limite de cem partículas evita acumular efeitos. O aviso na tela e a mudança na barra complementam a informação. As partículas são visuais: quem calcula a perda de conservação é a regra da corrida.

10. Evolução do projeto (55 segundos)

Começamos com a concepção e um protótipo de formas simples. O histórico registra a corrida, o salto e as primeiras colisões antes das melhorias visuais. Depois entraram as camadas do cenário, os quadros e as partículas. Os testes no Windows mostraram que ampliar a janela deixava a imagem pouco definida. A solução foi desenhar na resolução final. Também corrigimos o rosto do cliente e adicionamos a música. As imagens mostram esses estágios, e os commits e o diário registram as mudanças.

Falas sugeridas dos slides 11 a 14

11. O que precisou de ajuste (50 segundos)

Os testes ajudaram a encontrar problemas visíveis. Algumas setas apareciam como retângulos, então as instruções passaram a usar palavras. A imagem ficava pouco nítida em tela grande, então o desenho passou a usar a resolução final. O cliente frontal parecia ter um olho só e recebeu um rosto com dois olhos e sorriso central. Também verificamos regras, telas e reinício por testes automáticos. Daniel jogou no Windows e aprovou o visual e o áudio. O próximo cuidado é ensaiar no computador da avaliação.

12. Participação da equipe e da IA (45 segundos)

Daniel, John e Lucas participaram das escolhas do tema, da corrida lateral, dos obstáculos e dos itens. Daniel executou os testes no Windows e relatou os problemas usados nas revisões. ChatGPT e Codex geraram grande parte do código e apoiaram os desenhos, os testes automatizados e a documentação. Esse uso está registrado com finalidade, partes afetadas e validação. A nossa responsabilidade na apresentação é compreender as regras e explicar as técnicas do projeto.

13. Materiais e créditos (30 segundos)

A principal referência foi o enunciado do professor. O jogo usa Python e Pygame, e a arte é desenhada no código. A música é House In a Forest Loop, de HorrorPen, com licença Creative Commons Attribution três ponto zero. Os três repositórios sugeridos estão registrados, mas não copiamos código deles. Os créditos completos e o registro de IA acompanham o projeto.

14. Demonstração do jogo (2 a 3 minutos, após exposição e perguntas)

Agora vamos mostrar o jogo funcionando. Começamos pelo menu, iniciamos a corrida e mostramos salto, gelo e obstáculos. Durante a partida, apontamos o HUD e as camadas do cenário. Pausamos uma vez, abrimos Áudio e retomamos. No resultado, mostramos vitória ou derrota e a opção de jogar novamente. Se perdermos, explicamos a condição e reiniciamos. As imagens da apresentação ajudam a mostrar efeitos que não apareçam nessa partida.

Perguntas que vale ensaiar

O que é o loop?

É a repetição de ler comandos, atualizar os dados e desenhar. Está em game/app.py, na função main.

Como as colisões funcionam?

O jogador e os objetos têm retângulos. pygame.Rect.colliderect verifica se eles se sobrepõem. Isso é AABB. A caixa do cone aproxima sua forma.

Por que usar tempo no movimento?

O deslocamento depende da velocidade multiplicada pelo tempo. App atualiza Run em passos fixos de 1/60 s, mantendo a simulação consistente entre taxas de desenho diferentes.

Como o salto volta ao chão?

O salto começa com velocidade para cima. A gravidade muda a velocidade, e o chão limita a posição e libera o próximo salto.

Como a câmera funciona?

Os objetos ficam em posições do mundo. Para desenhar, o jogo subtrai a distância da câmera. O HUD fica em coordenadas fixas de tela.

O que muda ao apertar F11?

Muda a resolução do desenho, mantendo a proporção. As coordenadas da física e das colisões continuam iguais.

Quais são as três técnicas gráficas?

Parallax: camadas com velocidades diferentes. Animação por quadros: troca de poses. Partículas: efeitos que se movem e desaparecem.

O que os testes automáticos verificam?

Regras, estados, interface, resolução, efeitos e controle de áudio. Usam dispositivos simulados e não comprovam FPS real ou experiência de uma pessoa jogando.

Qual foi o papel da equipe e da IA?

A equipe participou da concepção e das escolhas do jogo. Daniel testou no Windows e relatou problemas. A IA gerou grande parte da implementação e apoiou as revisões e os materiais.

Demonstração e fontes

Demonstração em 2 a 3 minutos

1. Mostrem o menu e iniciem a corrida.
2. Joguem a rota, destacando salto, coleta, obstáculos e HUD.
3. Pausem uma vez e abram Áudio. Mostrem que a corrida não avança.
4. Retomem e mostrem o resultado, reinício ou retorno ao menu.
5. Se houver derrota, expliquem a condição e reiniciem. Não é preciso vencer e perder na mesma demonstração curta.
6. Se um efeito não aparecer nessa partida, usem o slide para explicá-lo.

Onde localizar o código

game/app.py: App.handle_event, App.update e main.
game/run.py: Run.update decide coleta, dano e resultado.
game/entities.py: Player.update e rect.
game/parallax.py: Parallax.draw e layer_offset.
game/animation.py: CourierAnimation.frame_index.
game/particles.py: Effects.update e draw.

Origem das imagens

A comparação de evolução usa duas capturas de Daniel: protótipo e revisão da renderização nativa anterior à correção do cliente. A terceira imagem e as demais cenas atuais vêm do renderizador do próprio código, com valores preparados para ilustrar recursos. A sequência do entregador mostra os quadros reais gerados pelo jogo. Essas imagens não são registros de partidas completas nem medições de desempenho.

Fontes e créditos

Enunciado do professor Pedro Cândido, especialmente seções 8 e 11. Código e documentos de Danielpr614/corre-que-derrete, com registros de 07/10 e 15/10/2026.
Python: python.org. Pygame: pygame.org.
Música: House In a Forest Loop, HorrorPen, CC BY 3.0. Origem: opengameart.org/content/loop-house-in-a-forest. Licença: creativecommons.org/licenses/by/3.0/.
ChatGPT/Codex, OpenAI: assistência à implementação e aos materiais. Créditos completos no repositório.